UAV Design & Simulation Internship

Task – 2

Title: Flight Dynamics + PID Control System Implementation

1. UAV Flight Dynamics:

A quadcopter's flight controller translates the pilot's commands (or autonomous waypoint directives) into specific motor speed differentials to manipulate the drone along its lateral, longitudinal, and vertical axes.

- **Altitude and Throttle Dynamics**

Throttle is the fundamental mechanism controlling the total vertical lift of the aircraft. It is commanded by adjusting the total thrust of all four motors uniformly. When the cumulative aerodynamic thrust vector pointing upward along the body's z-axis exceeds the total gravitational force vector (mg) pointing downward, the drone achieves positive vertical acceleration and climbs. Conversely, reducing the uniform RPM allows gravity to overpower the thrust, causing a controlled descent. To maintain a perfectly stable hover, the flight controller must modulate the motors such that the total thrust exactly equals the mass of the drone multiplied by the acceleration due to gravity, while compensating for minute fluctuations in air density and battery voltage drops.

- **Roll (Lateral Deflection)**

Roll refers to the tilting of the quadcopter around its longitudinal x-axis, facilitating lateral strafing motion to the left or right. This movement is governed by creating a thrust differential between the left and right motor pairs. If the control system increases the angular velocity of the motors on the right side while proportionally decreasing the angular velocity of the motors on the left side, the total vertical thrust remains constant (preventing altitude loss), but a net torque is generated along the x-axis.

Assuming motors 1 and 3 reside on the lateral roll axis, and L represents the physical distance from the drone's center of mass to the center of the propeller, the roll torque is derived mathematically as :

$$\tau_\phi = L(k\omega_1^2 - k\omega_3^2)$$

As the drone rolls to the left, a portion of its vertical thrust vector is redirected horizontally, causing the vehicle to accelerate laterally through the air.

- **Pitch (Longitudinal Deflection)**

Pitch refers to the tilting of the drone around its lateral y-axis, which enables forward and backward flight. The physical mechanics of pitch are identical to roll, merely rotated by 90 degrees. Pitch is controlled by creating a thrust differential between the front and rear motor pairs. Increasing the RPM of the rear motors while slowing the front motors generates a torque that tips the nose of the quadcopter downward.

Assuming motors 2 and 4 reside on the longitudinal pitch axis, the pitch torque is expressed as :

$$\tau_\theta = L(k\omega_2^2 - k\omega_4^2)$$

As the drone pitches forward, the total thrust vector leans forward, overcoming the drone's horizontal inertia and aerodynamic drag to propel the aircraft forward.

- **Yaw (Vertical Axis Rotation)**

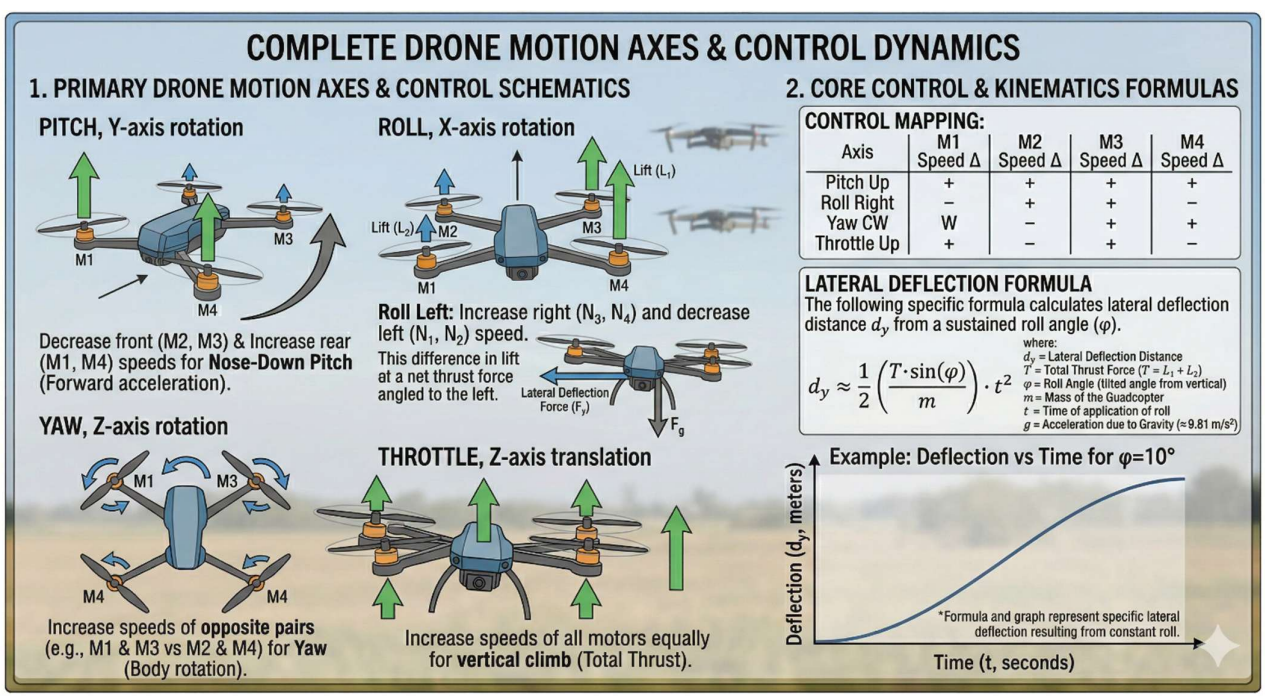
Yaw is the rotation of the drone around its vertical z-axis, allowing it to change its heading without altering its spatial coordinates. Unlike roll and pitch, which rely on asymmetric thrust to tilt the drone, yaw relies on the conservation of angular momentum and the manipulation of aerodynamic drag torque.

To achieve a net-zero yaw torque during a stable hover, a quadcopter features two motors situated on opposing diagonals that rotate clockwise, while the other diagonal pair rotates counter-clockwise. Because the drag torque acts in the direction opposite to propeller rotation, the torques naturally cancel each other out when all motors spin at the same speed. To execute a yaw maneuver, the flight controller speeds up one diagonal pair of motors while simultaneously slowing down the other pair. This maintains the overall vertical thrust (keeping altitude constant) but creates a severe imbalance in the reactive drag torques, forcing the drone's chassis to spin in the direction of the slower-spinning motor pair.

The total yaw torque acting on the body frame is calculated by summing the drag torques of the individual motors, accounting for their rotation direction :

$$\tau_\psi = b(\omega_1^2 - \omega_2^2 + \omega_3^2 - \omega_4^2)$$

Flight Axis	Physical Actuation Mechanism	Mathematical Torque Representation
Throttle (Z-Axis)	Uniform adjustment of all four motor velocities.	$T = \sum_{i=1}^{4} k\omega_i^2$
Roll (X-Axis)	Thrust differential between left and right motor pairs.	$\tau_\phi = L(k\omega_1^2 - k\omega_3^2)$
Pitch (Y-Axis)	Thrust differential between front and rear motor pairs.	$\tau_\theta = L(k\omega_2^2 - k\omega_4^2)$
Yaw (Z-Axis)	Drag torque differential between diagonal motor pairs.	$\tau_\psi = b(\omega_1^2 - \omega_2^2 + \omega_3^2 - \omega_4^2)$



Real-World Industry Applications of UAV Flight Dynamics

The theoretical physics governing rigid-body dynamics are not merely academic; they are the functional prerequisites for massive commercial enterprises and advanced hardware engineering. The precise manipulation of pitch, roll, yaw, and throttle enables transformative operational benefits across global logistics and high-end cinematography.

Automated Logistics and Delivery Networks (Zipline)

Commercial delivery drones have evolved entirely beyond remote-controlled aerial cameras; they are now defined by their payload capabilities, autonomous mission logic, and precise aerodynamic control. Companies like Zipline have revolutionized urban and rural delivery networks by treating individual drone routes like a miniature airline system with precision scheduling.

Zipline's operations heavily utilize advanced flight dynamics to meet strict delivery cycle times. Their Platform 2 (P2) system is a highly complex aircraft that must autonomously transition between vertical multi-rotor hovering and fixed-wing forward cruising. During the delivery phase, the drone must utilize precise pitch and roll adjustments to navigate densely populated urban corridors, calculate dynamic flight path recalculations in real-time to avoid obstacles, and counteract aerodynamic drag to maintain forward velocity.

Furthermore, to combat wind shear during the critical payload drop phase, the flight controller must rapidly calculate the required motor RPM differentials to maintain a perfectly stable hover while an autonomous delivery unit (the "Droid") is lowered on a tether. The success of these autonomous supply chains—which have completed millions of deliveries and maintained 85% to 95% on-time performance metrics in critical healthcare sectors—is entirely dependent on the flawless programmatic execution of multi-axis flight dynamics.

Advanced Cinematography and Camera Stabilization (DJI Gimbals)

The exact same physics that govern the attitude of a massive multi-rotor UAV are miniaturized and inverted to create 3-axis camera gimbals. In professional cinematography and industrial mapping, a drone's camera must remain perfectly level to capture usable data, irrespective of the violent aerodynamic corrections the drone's chassis is undergoing above it.

UAV camera systems encounter immense dynamic perturbations. Normal flight operations induce vibrations ranging from 0.5Hz to 3Hz, while aggressive maneuvers can spike perturbations up to 20Hz. Furthermore, the natural resonance frequencies of carbon fiber drone airframes typically range from 5Hz to 15Hz, while the rapid spinning of the BLDC motors creates severe high-frequency vibrations between 50Hz and 200Hz.

To eliminate motion blur and ensure sub-degree stabilization precision, mechanical gimbals (such as those developed by DJI) act as inverse flight controllers. An internal Inertial Measurement Unit (IMU) detects the drone's pitch, roll, and yaw movements in milliseconds. The gimbal's internal processor calculates the exact inverse angular velocity required and commands miniature brushless motors to rotate the camera payload on three axes. If the drone pitches forward aggressively to accelerate, the gimbal's pitch motor rotates the camera upward at the exact same angular rate, maintaining a perfectly level horizon. This application of dynamic stabilization relies on internal cascaded PID controllers tailored specifically to motor stiffness, payload mass strength, and gyro noise filtering.

Proportional-Integral-Derivative (PID) Control Theory

As established, quadcopters are severely underactuated and rely on highly non-linear dynamics. Consequently, open-loop control—where a system executes commands without checking the results—is functionally impossible for stable flight. To maintain equilibrium against gravity, wind, and inertia, drones utilize a closed-loop feedback mechanism known as the Proportional-Integral-Derivative (PID) controller. This algorithmic structure automatically compares the user's desired target value (the setpoint) with the actual measured state of the system (the process variable), calculates the difference as an error value $e(t)$, and applies corrective mathematical actions to drive the error to zero.

The continuous-time mathematical equation for a standard PID controller is defined as:

$$u(t) = k_p e(t) + k_i \int_0^t e(\tau)d\tau + k_d \frac{de(t)}{dt}$$

where $u(t)$ represents the final control signal sent to the actuators (the electronic speed controllers commanding the motors), and K_p, K_i, and K_d are the tunable gain coefficients. Each mathematical component serves a highly specific role in shaping the system's dynamic step response.

The Proportional Term (P-Gain)

The proportional component responds exclusively to the current, instantaneous error value. It produces a control output that is directly proportional to the magnitude of the error (K_p X e(t)). Functionally, the P-gain acts as the primary determinant of the quadcopter's flight feel and responsiveness.

When a pilot commands a sharp pitch forward, a large error is generated. A higher K_p value forces the drone to accelerate its motors aggressively to reach the target angle, thereby drastically reducing the system's "rise time". However, increasing the proportional gain introduces a critical trade-off. If K_p is set too high, the drone will accelerate so violently that its physical momentum carries it past the desired setpoint. This phenomenon is known as overshoot. Once the drone overshoots, the error becomes negative, forcing the P-controller to command an immediate reverse thrust. This over-correction cycle results in rapid, high-frequency oscillations (often described as a fast "buzz" or violent shaking), which severely destabilizes the aircraft. Conversely, if K_p is too low, the quadcopter will feel sluggish, sloppy, and unresponsive, drifting aimlessly because it lacks the control authority to reach the setpoint. Importantly, a pure P-controller will never fully eliminate steady-state error; as the drone approaches the setpoint and the error shrinks, the proportional thrust also shrinks, resulting in an asymptotic approach that never quite reaches the exact target.

The Integral Term (I-Gain)

The integral component addresses the limitations of the proportional term by maintaining a mathematical "memory" of the system. It continuously calculates the cumulative sum of all past errors over time $\oint e(\tau)d\tau$. The primary objective of the I-term is the total elimination of steady-state error.

In a real-world scenario, if a drone is commanded to hover at 10 meters but is carrying a heavy payload or fighting a constant downward wind, the P-term alone might stabilize the drone at 9.5 meters, lacking the mathematical "push" to close the final 0.5-meter gap because the error is too small to generate sufficient thrust. The I-term recognizes that this 0.5-meter error is persisting over time. As the seconds tick by, the integral accumulator grows larger and larger, steadily ramping up the motor RPM until the drone is forced exactly to the 10-meter mark.

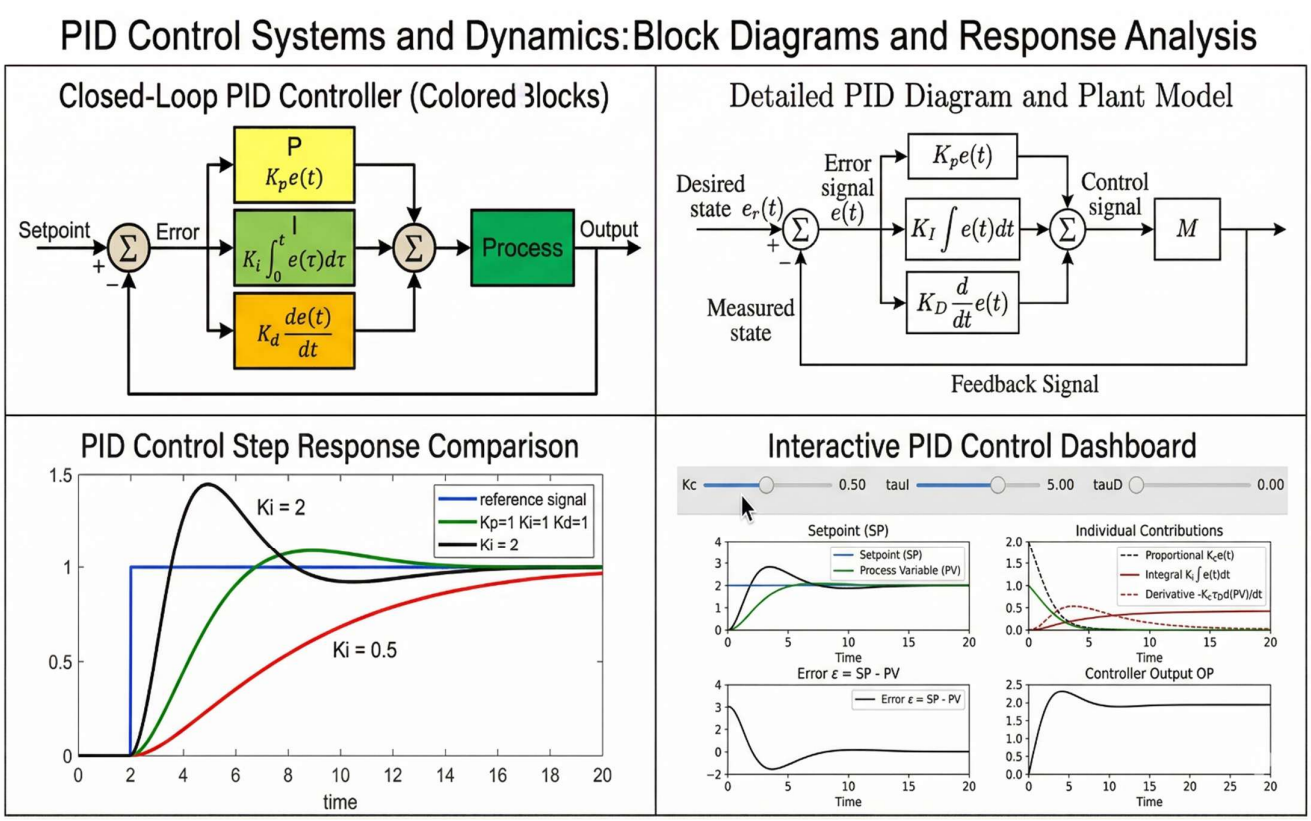
While essential for long-term accuracy and tracking sweeping maneuvers in windy conditions, the I-term introduces complex dynamic risks. Because it relies on historical accumulation, it is inherently slow to respond. If the K_i gain is set too high, the controller will accumulate a massive sum during the initial climb. By the time the drone reaches the target, the bloated integral term will force the drone to severely overshoot the target, leading to increased settling times and slow, pendulum-like, low-frequency wobbles as it attempts to "unwind" the accumulated mathematics.

The Derivative Term (D-Gain)

The derivative component acts as the system's predictive element. By assessing the instantaneous rate of change of the error $\frac{de(t)}{dt}$,the D-term anticipates the future state of the drone based on its current trajectory and velocity.

Functionally, the derivative term acts as an algorithmic shock absorber or damper. When the drone is rapidly pitching toward its setpoint, the error is decreasing quickly, resulting in a large negative derivative. The D-term multiplies this negative rate by the K_d gain to subtract thrust from the overall control signal. This "braking" effect prevents the aggressive P-term from overshooting the target and neutralizes bounce-back oscillations, vastly reducing the system's settling time.

However, the derivative term is incredibly sensitive to noise. Because it reacts to instantaneous changes, high-frequency vibrations from the drone's carbon fiber frame and spinning motors are interpreted as rapid error fluctuations. If K_d is set too high, the flight controller will command violent, microscopic motor stuttering to fight this phantom noise. These stuttering forces the Electronic Speed Controllers (ESCs) to rapidly switch extreme currents, which severely degrades flight efficiency and can cause the motors to overheat to the point of structural failure and mid-air fires.



The Problem:

Controller Tuning Parameters:

- Proportional Gain (K_p): **2.0**
- Integral Gain (K_p): **1.0**
- Derivative Gain (K_d): **0.5**
- Time Step (Δt): **0.1 seconds** (The controller loop runs 10 times per second)

Current System State:

- Target Setpoint (SP): **50 °C**
- Current Process Variable/Temperature (PV): **45 °C**
- Previous Error (e_{t-1} from the last loop): **3 °C**
- Accumulated Integral Sum from previous steps: **15.0**

Task: Calculate the total Controller Output (OP) for the current time step.

Step-by-Step Solution

1. Calculate the Current Error (e_t)

The error is the difference between where you want to be and where you are.

- e_t = SP - PV
- e_t = 50 - 45
- **e_t = 5**

2. Calculate the Proportional Term (P)

The P-term provides an output directly proportional to the current error.

- P = K_p X e_t
- P = 2.0 X 5
- **P = 10.0**

3. Calculate the Integral Term (I)

The I-term accumulates past errors over time. First, we update the accumulated integral sum, then multiply by the Integral Gain.

- *New Integral Sum* = Previous Integral Sum + (e_t X Δt)
- *New Integral Sum* = 15.0 + (5 X 0.1) = 15.0 + 0.5 = 15.5
- I = K_i X New Integral Sum
- I = 1.0 X 15.5
- **I = 15.5**

4. Calculate the Derivative Term (D)

The D-term looks at the rate of change of the error (predicting future error).

- *Rate of Change* = $\frac{e_t - e_{t-1}}{\Delta t}$
- *Rate of Change* = $\frac{5-3}{0.1} = \frac{2}{0.1}$ = 20
- D = K_d X {Rate of Change}
- D = 0.5 X 20
- **D = 10.0**

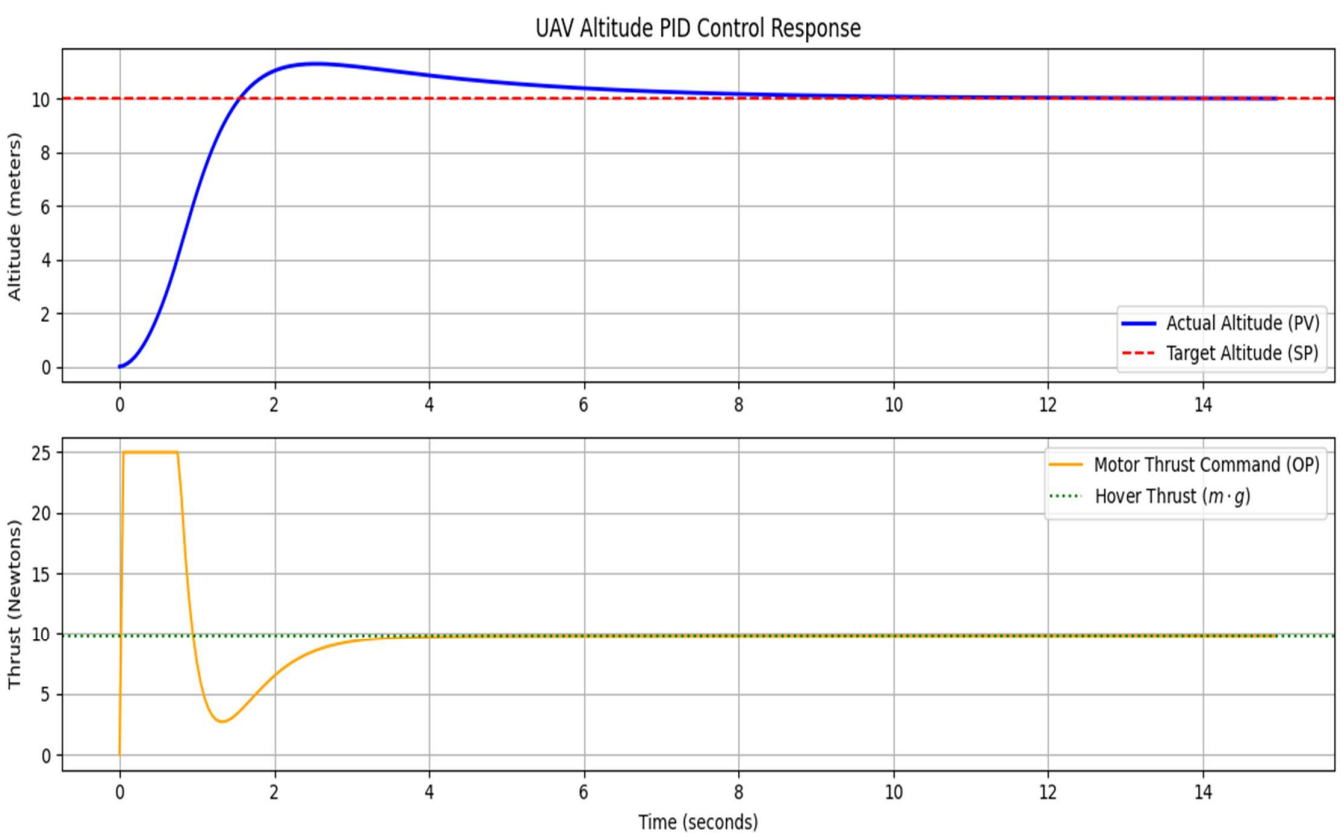
5. Calculate the Total Controller Output (OP)

Sum the three terms to get the final control signal sent to the heater.

- OP = P + I + D
- OP = 10.0 + 15.5 + 10.0
- **Total Output (OP) = 35.5**

PID Control(Python Simulation)

Output:



How This Simulation Works

1. **The Plant Dynamics:** The code uses basic Newtonian physics. At every time step Δt, it calculates the net force on the system: $F_{net} = T - mg - cv$. It then derives the acceleration, updates the velocity, and finally updates the altitude.
2. **The I Term at Work:** Notice the green dotted line in the bottom graph generated by the code. That represents the exact thrust required to hover (m g). The PID controller doesn't explicitly "know" the mass of the vehicle. Instead, the Integral (K_i) term slowly accumulates the steady-state error until the output naturally settles precisely at the hover thrust limit.
3. **Saturation Constraints:** Real actuators have limits. The output_limits = (0, 25) mimics the physical limitations of the motors.

The Tuning Sequence for Stable Hover

If you were tuning this on a test stand, the standard methodology yields these sequential observations:

1. **Zero everything out:** Set I and D to zero.
2. **Push P until it shakes:** Increase K_p until the drone achieves lift off but starts oscillating rapidly (underdamped).
3. **Dampen with D:** Increase K_d to act as a shock absorber until those rapid P-oscillations smooth out. The drone should now snap to altitudes quickly without bouncing, but it might still have a steady-state error.
4. **Lock it in with I:** Slowly increase K_i just enough so that the drone gently settles perfectly onto the target altitude without introducing slow bobbing.

To see exactly how these math parameters translate to flight dynamics, try experimenting with the values in this interactive hover simulator:

Thank you……

The End……